

Dokumentation

ToDo-Applikation mit Git, GitHub und Docker

Name: Joel Kapoor

Datum: 16.09.2026

GitHub-Repository: <https://github.com/joelkapoor720/docker-nodejs-joel>



1. Repository klonen

Das Repository wurde von GitHub geklont und lokal geöffnet.

```
git clone https://github.com/joelkapoor720/docker-nodejs-joel.git
```

```
cd /c/Users/joelk/Documents/docker-nodejs-joel
```

Die Arbeit wurde unter Windows mit **Git Bash** durchgeführt.

```
joelk@DESKTOP-KODIKDD MINGW64 ~/Documents/docker-nodejs-joel (main)
$ git remote -v
origin https://github.com/joelkapoor720/docker-nodejs-joel.git (fetch)
origin https://github.com/joelkapoor720/docker-nodejs-joel.git (push)
```

2. README.md

Die README.md wurde erstellt bzw. überarbeitet. Sie enthält die wichtigsten Anweisungen zur Installation und zum Starten der Anwendung mit Docker.

```
joelk@DESKTOP-KODIKDD MINGW64 ~/Documents/docker-nodejs-joel (main)
$ cat README.md
# Docker Node.js ToDo-App

Eine einfache ToDo-Anwendung mit Node.js, die in einem Docker-Container ausgeführt wird.

## Installation

### 1. Repository klonen

Repository von GitHub klonen:



```
bash
git clone https://github.com/joelkapoor720/docker-nodejs-joel.git
cd docker-nodejs-joel
```



### 2. Abhängigkeiten installieren

Node.js muss installiert sein.

Die benötigten Abhängigkeiten werden mit folgendem Befehl installiert:



```
bash
npm install
```



### 3. Docker installieren

Installiere Docker Desktop und stelle sicher, dass Docker Desktop läuft.

Überprüfe die Docker-Installation mit:



```
bash
docker --version
```



Zusätzlich kann Docker mit folgendem Befehl getestet werden:



```
bash
docker run hello-world
```



### 4. Docker-Image erstellen

Im Projektordner wird das Docker-Image mit dem 'Dockerfile' erstellt:



```
bash
```


```

```
docker build -t docker-nodejs-joel .

### 5. Docker-Container starten

Starte anschließend einen Container mit:



```
bash
docker run --name todo-app -p 3000:3000 docker-nodejs-joel
```



Die Anwendung läuft danach im Docker-Container.

### 6. Anwendung öffnen

Öffne im Browser:



```
text
http://localhost:3000
```



Dort kann die ToDo-Anwendung verwendet werden.

### 7. Laufenden Container überprüfen

Mit folgendem Befehl kann überprüft werden, ob der Container läuft:



```
bash
docker ps
```



Der Container sollte den Namen 'todo-app' haben und den Port '3000' verwenden.

### 8. Container stoppen

Um den Container zu stoppen:



```
bash
docker stop todo-app
```



Um den Container später wieder zu starten:



```
bash
docker start todo-app
```


```

3. Änderungen mit Git speichern

Die Änderungen wurden überprüft und für den Commit vorgemerkt:

```
git status
```

```
git add README.md
```

Danach wurde ein Commit erstellt:

```
git commit -m "Update README for Git Bash"
```

```
joelk@DESKTOP-KODIKDD MINGW64 ~/Documents/docker-nodejs-joel (main)
$ git log --oneline -3
c6f5f0d (HEAD -> main, origin/main, origin/HEAD) Update README for Git Bash
7b20861 Dockerize Node.js application
0e8f648 Correct Docker initialization step in README
```

4. Änderungen zu GitHub übertragen

Der Commit wurde zu GitHub übertragen:

```
git push origin main
```

Anschließend wurde überprüft, ob das Repository aktuell ist:

```
git status
```

Da ich dies bereits tat, war folgendes das Ergebnis

```
Everything up-to-date
```

```
joe1k@DESKTOP-KODIKDD MINGW64 ~/Documents/docker-nodejs-joe1 (main)
$ git push origin main
Everything up-to-date
```

5. Dockerfile erstellen

Für die Containerisierung wurde in VS Code ein Dockerfile erstellt:

```
FROM node:24-slim
```

```
WORKDIR /app
```

```
COPY package*.json ./
```

```
RUN npm install
```

```
COPY . .
```

```
EXPOSE 3000
```

```
CMD ["node", "src/index.js"]
```

```
joe1k@DESKTOP-KODIKDD MINGW64 ~/Documents/docker-nodejs-joe1 (main)
$ cat Dockerfile
FROM node:24-slim

WORKDIR /app

COPY package*.json ./

RUN npm install

COPY . .

EXPOSE 3000

CMD ["node", "src/index.js"]
```

6. .dockerignore

Zusätzlich wurde eine .dockerignore erstellt. Dadurch werden unnötige Dateien wie node_modules oder das Git-Verzeichnis beim Docker-Build ausgeschlossen.

```
node_modules
```

```
npm-debug.log
```

```
.git
```

```
.gitignore
```

```
Dockerfile
```

```
.dockerignore
```

```
joelk@DESKTOP-KODIKDD MINGW64 ~/Documents/docker-nodejs-joel (main)
$ cat .dockerignore
node_modules
npm-debug.log
.git
.gitignore
Dockerfile
.dockerignore
```

7. Docker-Image erstellen

Das Docker-Image wurde mit folgendem Befehl erstellt:

`docker build -t docker-nodejs-joel .`

Der Build wurde erfolgreich abgeschlossen.

```
joelk@DESKTOP-KODIKDD MINGW64 ~/Documents/docker-nodejs-joel (main)
$ docker build -t docker-nodejs-joel .
[+] Building 1.6s (11/11) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 175B
=> [internal] load metadata for docker.io/library/node:24-slim
=> [auth] library/node:pull token for registry-1.docker.io
=> [internal] load .dockerignore
=> => transferring context: 112B
=> [1/5] FROM docker.io/library/node:24-slim@sha256:2fe369e969550cde8e867afc3fe370b260140cab4a23d467074295b42163d553
=> => resolve docker.io/library/node:24-slim@sha256:2fe369e969550cde8e867afc3fe370b260140cab4a23d467074295b42163d553
=> [internal] load build context
=> => transferring context: 4.20kB
=> CACHED [2/5] WORKDIR /app
=> CACHED [3/5] COPY package*.json ./
=> CACHED [4/5] RUN npm install
=> [5/5] COPY . .
=> exporting to image
=> exporting layers
=> exporting manifest sha256:1df51d5a475a6272fc07d48bf1fb65d2631292b4c9bedb963726b82deda53387
=> exporting config sha256:f64e4ef0ed28913e2ab7c782b2e01e610635aacc2d719131b965abfaa73cf6fb
=> exporting attestation manifest sha256:4f722bd5a4dcca1b7191432c34047ed6adfc0002c09b13c13c89cd6e7184d137
=> exporting manifest list sha256:b870654ecf1fb3ed6779467bef66d5235c86f9c4a2f376fdd12edb5af745ef65
=> naming to docker.io/library/docker-nodejs-joel:latest
=> unpacking to docker.io/library/docker-nodejs-joel:latest
```

8. Docker-Container starten

Anschließend wurde die Anwendung in einem Docker-Container gestartet:

`docker run --name todo-app -p 3000:3000 docker-nodejs-joel`

Die Anwendung meldete:

Listening on port 3000

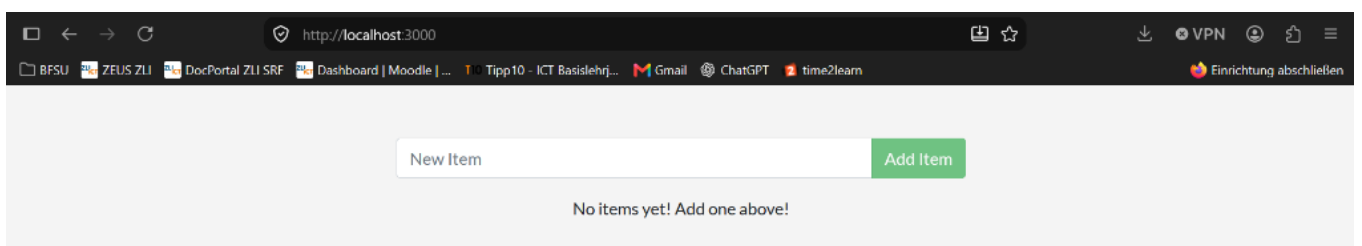
```
joelk@DESKTOP-KODIKDD MINGW64 ~/Documents/docker-nodejs-joel (main)
$ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS                               NAMES
2876a26b157f   docker-nodejs-joel   "docker-entrypoint.s..." 4 hours ago   Up 32 minutes   0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp   todo-app
88afb2c7d243   hello-world      "/hello"                 4 hours ago   Exited (0) 4 hours ago                               epic_aryabhata
0440db9a1bfa   hello-world      "/hello"                 6 hours ago   Exited (0) 6 hours ago                               amazing_cerf
```

9. Anwendung testen

Die Anwendung wurde im Browser geöffnet:

`http://localhost:3000`

Die ToDo-Anwendung wurde erfolgreich angezeigt.



10. Container überprüfen

Der laufende Container wurde mit folgendem Befehl überprüft:

`docker.exe ps`

Dabei wurde der laufende Container mit dem verwendeten Port angezeigt.

```
MINGW64/c/Users/joelk/Documents/docker-nodejs-joel

joelk@DESKTOP-KODIKDD MINGW64 ~
$ cd /c/Users/joelk/Documents/docker-nodejs-joel

joelk@DESKTOP-KODIKDD MINGW64 ~/Documents/docker-nodejs-joel (main)
$ docker.exe ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS                               NAMES
2876a26b157f   docker-nodejs-joel   "docker-entrypoint.s..." 4 hours ago   Up 28 minutes   0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp   todo-app
```

11. Abschluss

Die ToDo-Anwendung wurde erfolgreich mit Git und GitHub versioniert und mit Docker containerisiert. Das Docker-Image wurde erstellt und die Anwendung erfolgreich in einem Docker-Container ausgeführt.

Die Anwendung konnte im Browser geöffnet werden und die Änderungen wurden auf GitHub gespeichert.